

User Study

Instructions for Grading. **PLEASE READ CAREFULLY.**

Overview

Imagine you are a user interacting with a chatbot service (e.g., Gemini, ChatGPT, or DeepSeek) for help with programming tasks. On each page, you will see:

- **A Question:** This could be a vague user query or a code snippet requiring clarification or completion. Questions containing code will be displayed in code blocks for clarity.
- **Two Responses:** labeled Answer A and Answer B

Your Task

Please evaluate Answer A and Answer B based on the following criteria:

- **Precision and Focus:** How directly the response targets the question's core issue, using clear guidance, examples, or code.
- **Contextual Fit:** How well the response matches the user's programming goal (e.g., debugging, feature implementation).
- **Answer Faithfulness:** How effectively the response provides actionable steps or code to achieve the user's desired outcome.
- **Correctness:** Correctness in LLM responses refers to the accuracy and factual validity of the information, code, or guidance provided, ensuring it is free from errors and aligns with the user's query and intended outcome.

How to Rate

For each question, you will provide two types of ratings:

1. Preference Rating (1–5 scale):

- **"1"**: Answer A is better, based on the criteria above.
- **"5"**: Answer B is better, based on the criteria above.
- **"3"**: Both answers are equally good, based on the criteria above.

Note:

- **Incomplete Responses:** Some responses may appear cut off due to a 256-word limit imposed on the chatbots. Please grade only the content provided, ignoring any abrupt endings or non-English text (e.g., stray characters).
- **Code Formatting:** Questions and responses containing code will be presented in code blocks or artifact tags for clarity. Ensure you review these carefully for correctness and relevance.
- **Consistency:** Try to apply the same standards across all evaluations to ensure fair comparisons.

Question 1

Question:

I'm having trouble with my function.

Answer A:

“Extra context:

User Question: *I'm having trouble with my function.*

AI clarification question: *Could you tell me what language your function is written in and what the function is supposed to do? Also, what specific problem or error are you encountering?*

User follow up response: *My Java function for parsing JSON is throwing a 'NullPointerException'.*”

AI Final answer:

A `NullPointerException` when parsing JSON in Java usually means you are trying to access a property or method on a `null` object. Common causes include:

- The JSON string is empty or null.
- You are trying to access keys that don't exist without null checks.
- The JSON parsing library is not properly handling missing or malformed data.

To fix this, ensure your JSON string is valid and not null before parsing. Use null checks before accessing JSON properties. For example, if using Jackson:

```
JavaScript
ObjectMapper mapper = new ObjectMapper();
JsonNode root = mapper.readTree(jsonString);
if (root.has("key")) {
    String value = root.get("key").asText();
    // Use value
}
```

If you provide your code snippet, I can help identify the exact cause.

Answer B:

User: I'm having trouble with my function.

Model:

Please provide me with the function! I need to see the code to understand what's going wrong.



I'm here to help you debug it. Just paste the function here, and tell me:

- What is the function supposed to do?

- What is it actually doing?
- What is the error message you're seeing?
- What language is the function written in?

The more information you give me, the better I can assist you.

How to rate:

- "1": strongly prefer answer A
- "5": strongly prefer answer B
- "3": both answers are equally good

Precision and Focus (1-5): 1

Contextual Fit (1-5): 1

Answer Faithfulness (1-5): 1

Correctness (1-5): 1

Question 2

Question:

```
Python
def calculate_area(radius):
    return 3.14 * radius ** 2
# TODO: Handle negative radius
area = calculate_area(-5)
```

Answer A:

“Extra context:

User Question:

```
Python
def calculate_area(radius):
    return 3.14 * radius ** 2

# TODO: Handle negative radius
area = calculate_area(-5)
```

AI clarification question: What should the function do when the radius is negative?

User follow up response: It should return None for negative radius values.”

AI Final answer:

You're right to handle negative radius values. Here's an improved version of your function that returns **None** if the radius is negative:

```
Python
def calculate_area(radius):
    if radius < 0:
        return None
    return 3.14 * radius ** 2

area = calculate_area(-5)
print(area) # This will print None
```

This prevents incorrect area calculations for invalid input.

Answer B:

```
Python
def calculate_area(radius):
    """
    Calculates the area of a circle given its radius.

    Args:
        radius: The radius of the circle (a non-negative number).

    Returns:
        The area of the circle.
    """
    if radius < 0:
        return "Radius cannot be negative."
    return 3.14 * radius ** 2

area = calculate_area(-5)
print(area)
```

How to rate:

- "1": strongly prefer answer A
- "5": strongly prefer answer B
- "3": both answers are equally good

Precision and Focus (1-5): 1

Contextual Fit (1-5):1

Answer Faithfulness (1-5):1

Correctness (1-5):1

Question 3

```
Python
from flask import Flask
app = Flask(name)
@app.route('/')
# TODO: Add a view function
def main():
    return 'Hello World'
```

Answer A:

“Extra context:

User Question:

```
Python
from flask import Flask
app = Flask(__name__)
@app.route('/')
# TODO: Add a view function
def main():
    return 'Hello World'
```

AI clarification question: *What functionality do you want your Flask app to have? For example, do you want to collect user input, connect to a database, or something else?*

User follow up response: *I want to create a form to collect user emails and store them in a database.”*

AI Final answer:

Great! To create a Flask app that collects emails and stores them, you'll need to:

- Define a route for your form.
- Create an HTML template with a form.
- Handle form submission and save data to a database.

Here's a simple example:

```
Python
from flask import Flask, render_template, request, redirect
import sqlite3

app = Flask(__name__)

@app.route('/', methods=['GET', 'POST'])
def index():
    if request.method == 'POST':
        email = request.form['email']
        conn = sqlite3.connect('emails.db')
        cursor = conn.cursor()
        cursor.execute("INSERT INTO emails (email) VALUES (?)", (email,))
        conn.commit()
        conn.close()
        return redirect('/')
    return render_template('form.html')
```

You'll also need an HTML form (`form.html`) to collect emails.

Answer B:

```
Python
from flask import Flask
app = Flask(__name__)
@app.route('/')
def main():
    return 'Hello World'

if __name__ == '__main__':
    app.run(debug=True)
```

How to rate:

- "1": strongly prefer answer A
- "5": strongly prefer answer B
- "3": both answers are equally good

Precision and Focus (1-5): 1

Contextual Fit (1-5): 1

Answer Faithfulness (1-5): 1

Correctness: 1

Question 4

```
Python
class User:
    def init(self, name, age):
        self.name = name
        self.age = age
user = User('Alice', ) # TODO: specify age
```

Answer A:

“Extra context:

User Question:

```
Python
class User:
    def __init__(self, name, age):
        self.name = name
        self.age = age

user = User('Alice', ) # TODO: specify age
```

AI clarification question: What age should the *User* object have initially, and what is the purpose of the age attribute?

User follow up response: The age should be 25, used to track user age for a profile system.”

AI Final answer:

You can specify the age when creating the user object like this:

```
Python
class User:
    def __init__(self, name, age):
        self.name = name
        self.age = age

user = User('Alice', 25)
```

The **age** attribute will help track the user's age for profile-related features.

Answer B:

```
Python
class User:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

How to rate:

- "1": strongly prefer answer A
- "5": strongly prefer answer B
- "3": both answers are equally good

Precision and Focus (1-5): 1

Contextual Fit (1-5): 1

Answer Faithfulness (1-5): 1

Correctness (1-5): 1

Question 5

```
Python
while True:
    print('Running...')
    if not condition:
        break # TODO: Define condition
```

Answer A:

“Extra context:

User Question:

```
Python
while True:
    print('Running...')
    if not condition:
        break # TODO: Define condition
```

AI clarification question: What is the purpose of the loop, and what condition should stop it?

User follow up response: I want to stop the loop when a counter reaches 10.”

AI Final answer:

Here’s how you can implement that:

```
Python
counter = 0
while True:
    print('Running...')
    counter += 1
    if counter >= 10:
        break
```

This loop will print "Running..." ten times and then stop.

Answer B:

```
Python
while True:
    print('Running...')
    if not condition:
        break # TODO: Define condition
```

Explanation:

This code snippet is a simple while True loop that continuously prints "Running..." and then breaks out of the loop if the condition is false.

- while True: This creates an infinite loop. The code inside the loop will execute repeatedly forever unless a break statement is encountered.
- print('Running...'): This line prints the string "Running..." to the console.
- if not condition: This is the condition that will determine whether the loop should continue. not condition evaluates to True if condition is False and False if condition is True. This is a standard way to check for a boolean condition in Python.
- break: This statement immediately terminates the execution of the loop. When the if condition is True, the break statement is executed, causing the loop to stop.

How it works:

1. The code starts by printing "Running..." to the console.
2. It checks if...

How to rate:

- "1": strongly prefer answer A
- "5": strongly prefer answer B
- "3": both answers are equally good

Precision and Focus (1-5): 1

Contextual Fit (1-5): 1

Answer Faithfulness (1-5): 1

Correctness (1-5): 1